

Linear regression: Sonadora temperature and elevation example

An Bui

2026-05-15

Table of contents

Set up	1
Sonadora temperature example	2
a. Questions and hypotheses	2
b. Cleaning and summarizing	2
c. Exploratory data visualization	3
d. Temperature model	3
Diagnostics	4
Model summary	4
Generating predictions	5
Visualizing model predictions	5
Creating a table with model coefficients, 95% confidence intervals, and more . .	7

Set up

```
library(tidyverse) # general use
library(janitor) # cleaning data frames
library(here) # file/folder organization
library(ggeffects) # generating model predictions
library(flextable) # creating tables

# temperature data from Ramirez 2024
sonadora <- read_csv(here("data", "Temp_SonadoraGradient_Daily.csv"))
```

Sonadora temperature example

Data from Ramirez, A. 2024. Sonadora elevational plots: long-term monitoring of air temperature ver 877108. Environmental Data Initiative. <https://doi.org/10.6073/pasta/6b66eeca3092d8f2340b5132dec3> (Accessed 2025-05-14).

a. Questions and hypotheses

Question: Does elevation (in meters) predict temperature (in °C)?

H_0 : Elevation (m) does not predict temperature (°C).

H_A : Elevation (m) predicts temperature (°C).

b. Cleaning and summarizing

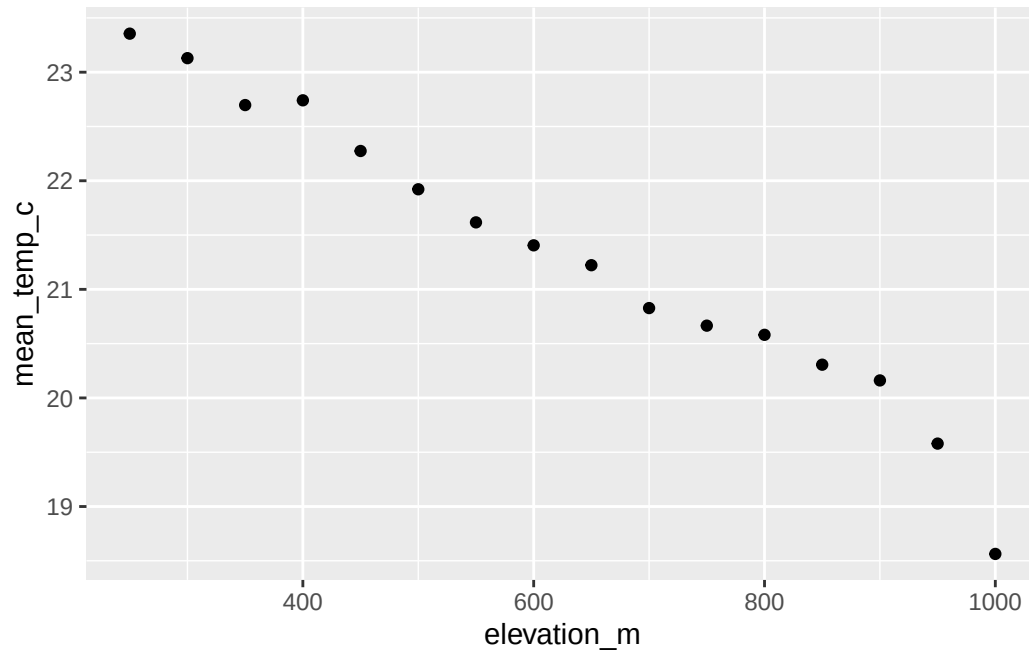
```
# creating new clean data frame
sonadora_clean <- sonadora |>
  # clean column names
  clean_names() |>
  # make the data frame longer
  pivot_longer(cols = plot_250:plot_1000,
               names_to = "plot_name",
               values_to = "temp_c") |>
  # separate plot name from elevation
  separate_wider_delim(cols = plot_name,
                      delim = "_",
                      names = c("plot", "elevation_m"),
                      cols_remove = FALSE) |>
  # remove plot column
  select(-plot) |>
  # make sure elevation is read as a number
  mutate(elevation_m = as.numeric(elevation_m))

# summarizing
sonadora_sum <- sonadora_clean |>
  # group by plot and elevation
  group_by(plot_name, elevation_m) |>
  # calculate mean temperature at each elevation
  summarize(mean_temp_c = mean(temp_c, na.rm = TRUE)) |>
  # undo the group_by function
```

```
ungroup() |>
# arrange in order of elevation
arrange(elevation_m)
```

c. Exploratory data visualization

```
# base layer: ggplot
ggplot(data = sonadora_sum,
       aes(x = elevation_m,
           y = mean_temp_c)) +
# first layer: points representing temperature at each elevation
geom_point()
```

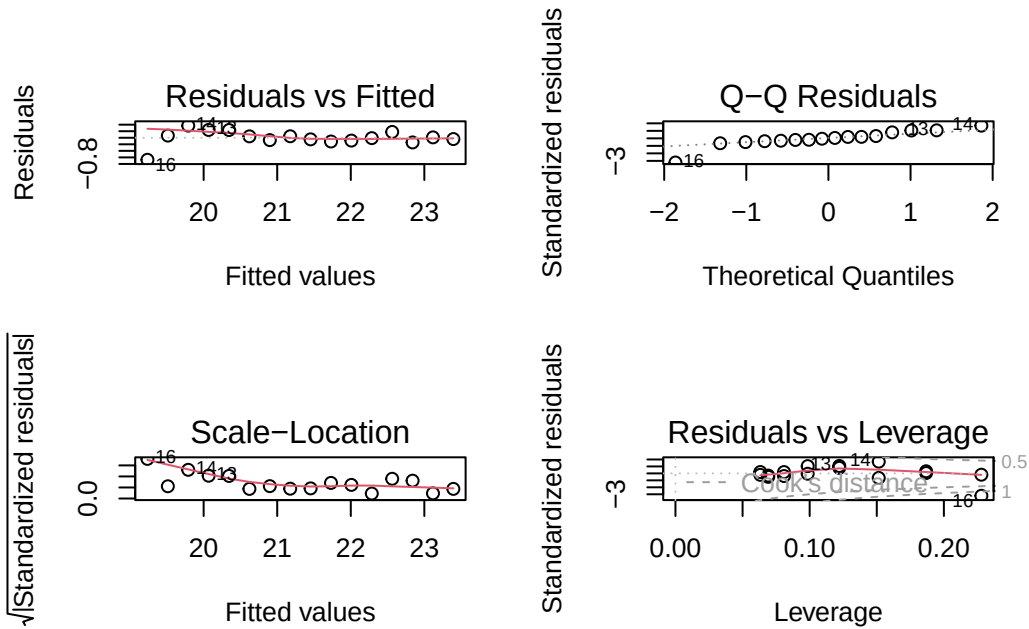


d. Temperature model

```
# model
temperature_model <- lm(
  mean_temp_c ~ elevation_m, # formula: response ~ predictor
  data = sonadora_sum # data frame
)
```

Diagnostics

```
# diagnostics
par(mfrow = c(2, 2))
plot(temperature_model)
```



Model summary

```
summary(temperature_model)
```

Call:

```
lm(formula = mean_temp_c ~ elevation_m, data = sonadora_sum)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.67288	-0.07577	0.00022	0.09393	0.37042

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	24.7807991	0.1719438	144.12	< 2e-16 ***

```
elevation_m -0.0055446  0.0002581  -21.48 4.08e-12 ***
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 0.238 on 14 degrees of freedom

Multiple R-squared: 0.9706, Adjusted R-squared: 0.9684

F-statistic: 461.4 on 1 and 14 DF, p-value: 4.075e-12

For each meter of elevation gain, you would expect a decrease in temperature equivalent to 0.01 ± 0.0003 (SE) °C.

Generating predictions

```
# model predictions
temperature_preds <- ggpredict(
  temperature_model,      # model object
  terms = "elevation_m"   # predictor (in quotation marks)
)

# calculate the temperature prediction at elevation = 900
ggpredict(
  temperature_model,      # model object
  terms = "elevation_m[900]" # predictor (in quotation marks) and predictor value in brackets
)
```

Predicted values of mean_temp_c

elevation_m	Predicted	95% CI
900	19.79	19.59, 19.99

At 900 m, the predicted temperature is 19.8 °C (95% CI: [19.6, 20.0]).

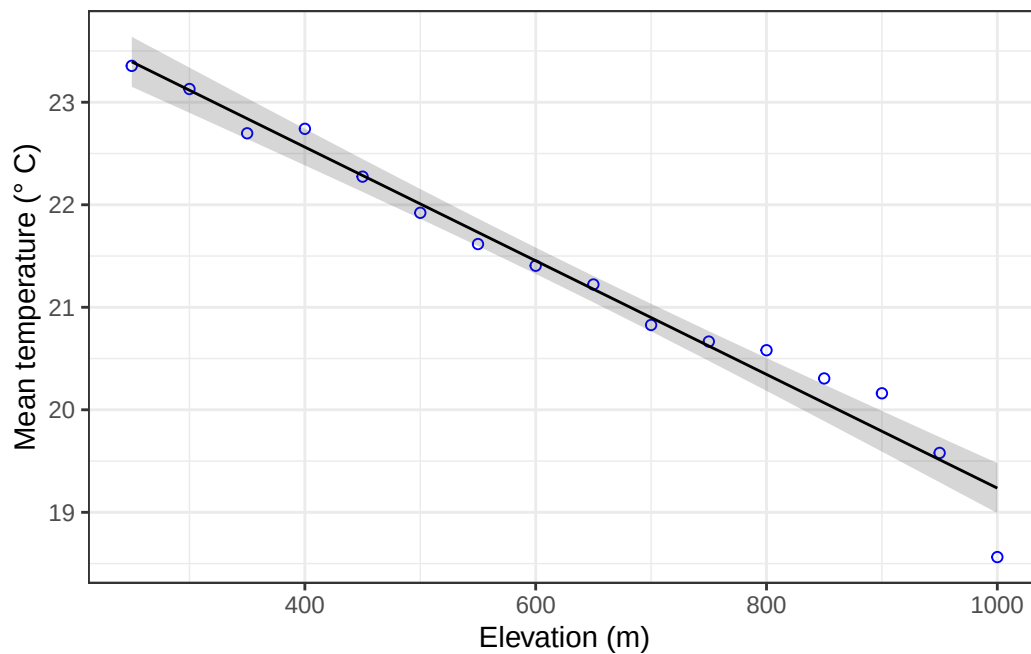
Visualizing model predictions

```
# base layer
ggplot(data = sonadora_sum,
  aes(x = elevation_m,
```

```

    y = mean_temp_c)) +
# first layer: temperature at each elevation
geom_point(color = "blue",
           shape = 21) +
# 95% CI ribbon
# uses model prediction data frame
geom_ribbon(data = temperature_preds,
           aes(x = x,
               y = predicted,
               ymin = conf.low,
               ymax = conf.high),
           alpha = 0.2) +
# model prediction line
# uses model prediction data frame
geom_line(data = temperature_preds,
          aes(x = x,
              y = predicted)) +
# axis labels
labs(x = "Elevation (m)",
     y = "Mean temperature (\u00B0 C)") +
theme_bw()

```



Creating a table with model coefficients, 95% confidence intervals, and more

```
as_flextable(temperature_model)
```

	Estimate	Standard Error	t value	Pr(> t)
(Intercept)	24.781	0.172	144.122	0.0000 ***
elevation_m	-0.006	0.000	-21.481	0.0000 ***

*Signif. codes: 0 <= '***' < 0.001 < '**' < 0.01 < '*' < 0.05*

Residual standard error: 0.238 on 14 degrees of freedom

Multiple R-squared: 0.9706, Adjusted R-squared: 0.9684

F-statistic: 461.4 on 14 and 1 DF, p-value: 0.0000

END OF SONADORA EXAMPLE